

Roteiro do Apresentador

Engenharia de IA, Agentes e Desenvolvimento Orientado a Especificação — 120 minutos, 68 slides

Para cada slide: a imagem, o tempo previsto e o texto a ser falado.

Capa

PALESTRA

Engenharia de IA e Desenvolvimento de Software Orientado a Especificação

Do modelo à spec: como times de software crítico incorporam IA sem perder auditabilidade.

120 minutos · desenvolvedores sêniores · sistemas de controle de infecção hospitalar (IRAS)

► Roteiro de fala

Boas-vindas e capa. Apresente-se em cerca de 20 segundos, citando seus 40 anos de engenharia — e não volte mais ao assunto; a autoridade já está estabelecida. Leia o subtítulo em voz alta, porque ele anuncia a tese: a palestra não é sobre 'a IA que escreve código', é sobre **como incorporar IA em software crítico sem perder auditabilidade**. A sigla IRAS — Infecções Relacionadas à Assistência à Saúde — é o domínio da plateia; use os exemplos deles a palestra inteira. (~30 s)

Agenda

BLOCO 0 · ABERTURA

Agenda: doze blocos, três pilares

Bloco	Tema	min
0	Abertura e agenda	3
1	Linha do tempo: do Perceptron aos agentes	3,5
2	MODELOS DE LINGUAGEM (LLMs) — arquitetura e panorama	14
3	O que mudou para quem escreve software	2
4	Engenharia de contexto e RAG	8
5	Agentes: anatomia, ferramentas, padrões, falhas	14
6	Protocolos e formatos de interoperabilidade	7,5
9	DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÃO (SDD)	19
11	OS SISTEMAS: LangNet, demonstração, AI Co.	15 5

1 LLMs

o motor: o que o modelo é e o que não é

2 SDD

o método: especificar e verificar

3 Sistemas

a prova: nossos sistemas rodando

Roteiro de fala

Agenda em uma tela. Mostre os doze blocos com o tempo ao lado, mas não leia todos — aponte para os **três pilares** à direita, que são o esqueleto da palestra: **LLMs** (o Bloco 2, o motor — o que o modelo de linguagem de grande porte realmente é), **SDD** (o Bloco 9, o método — Desenvolvimento de Software Orientado a Especificação, do inglês *Spec-Driven Development*) e **Sistemas** (o Bloco 11, a prova, com os nossos dois sistemas rodando). Diga que a agenda reaparece destacada na abertura dos blocos 5, 9 e 11 — é o mapa para a plateia não se perder em duas horas. (~1 min)

A tese

BLOCO 0 · ABERTURA

A tese central da palestra

“O gargalo deixou de ser escrever código. Passou a ser especificar e verificar.”

Capacidade virou mercadoria comum (commodity). Qualquer um chama uma API e gera código; o que separa os times é o método — não o acesso ao modelo.

Agente sem portão de verificação degrada de forma previsível. Sem um “gate” (portão que confere cada passo), a taxa de erro cai de um jeito que a matemática prevê — mostramos a curva no S28.

Especificação executável é o que torna código de IA auditável. Condição inegociável em software de saúde: sem spec verificável, não há como provar conformidade.

► Roteiro de fala

Este é o slide-âncora; ele reaparece no fechamento (S65). Leia a frase da tese devagar e deixe-a no ar por um segundo. Depois enuncie as três consequências como **promessas que você vai cumprir**: (1) gerar código virou mercadoria comum — qualquer um chama uma interface de programação; o que separa os times é o método de especificar e verificar. (2) Um agente sem *gate* — sem um portão que verifica cada passo — não falha de forma aleatória: ele degrada de um jeito que a matemática prevê, e mostro a curva no S28. (3) A especificação executável é o que transforma código de IA em algo auditável — e para quem vende software médico, auditabilidade não é luxo, é a diferença entre conformidade e não conformidade. Não desenvolva agora; só plante as três sementes. (~1 min)

Linha do tempo: Perceptron → agentes

BLOCO 1 · LINHA DO TEMPO

Do Perceptron ao desenvolvimento agêntico

ARQUITETURAS



"a mesma arquitetura, só que maior"

O QUE O DESENVOLVEDOR FAZIA



A ideia é de 1958. O que mudou foi compute e dado — e a unidade de trabalho de quem escreve software.

► Roteiro de fala

Leitura guiada — este é o slide que amarra a história. Gaste ~30 s indo de 1958 a 2012 na raia de cima e diga a frase que resume tudo: *'a ideia da rede neural é de 1958; o que mudou não foi a teoria, foi a capacidade de processamento e a quantidade de dados'*. Pare ~40 s na caixa de **2017, o Transformer** (destacada): a invenção que viabilizou tudo depois. Siga a seta curva até 2020 e diga *'a mesma arquitetura, só que maior'* — o GPT-3 não é uma ideia nova, é escala. Gaste ~40 s no salto de 2022 (o ChatGPT e o alinhamento a instruções). Então — e aqui está o argumento real, ~60 s — desça para a **raia de baixo: o que o desenvolvedor fazia**. A unidade de trabalho mudou cinco vezes: projetar características à mão, treinar o próprio modelo, fazer *ajuste fino* (fine-tune) sobre um modelo pronto, chamar uma interface de programação (API), e hoje — **orquestrar, especificar e verificar**. Feche apontando a última caixa: *'é aqui que estamos, e é sobre isso a palestra'*. Siglas: MLP = perceptron de múltiplas camadas; CNN = rede neural convolucional; LSTM = rede de memória de longo-curto prazo. (~3 min)

Onde a IA já está na área de vocês

BLOCO 1 · LINHA DO TEMPO

Onde a IA já está na área de vocês

ANTES

- escores de risco tabulares e preditivos de sepse
- redes convolucionais (CNN) em imagem médica
- alertas disparados por **regra fixa**

AGORA

- **extração** estruturada de texto clínico livre
- critérios de caso sobre a **evolução narrativa** do prontuário
- agentes com ferramentas sobre as **bases hospitalares**

O que mudou foi o acesso ao **texto livre do prontuário** — antes ilegível para a máquina, agora estruturável.

► Roteiro de fala

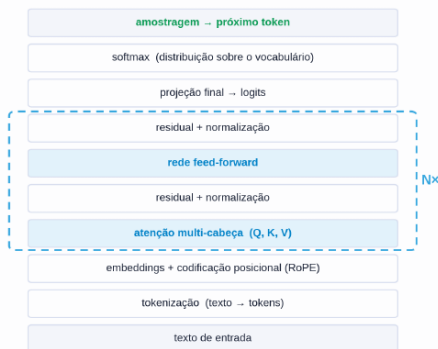
Aterrisse no domínio da plateia — rápido, 30 s. A IA já estava no hospital: escores de risco, preditivos de sepse, redes convolucionais lendo raio-X, alertas por regra. O que mudou de qualidade não foi 'ter IA' — foi passar a **ler o texto livre do prontuário**, que sempre foi a maior fonte de informação clínica e a mais inacessível para a máquina. Diga a frase de fecho: 'o que mudou foi o acesso ao texto livre' — e é isso que abre a porta para tudo que vem depois na palestra. (~30 s)

Anatomia do Transformer

BLOCO 2 · LLMS

Anatomia do Transformer

fluxo de baixo para cima — previsão do próximo token



Q, K, V (consulta, chave, valor). Cada token faz uma pergunta; todos anunciam o que têm; a semelhança decide de quem ele copia informação.

Multi-cabeça. Várias relações capturadas em paralelo, cada cabeça olhando um aspecto diferente.

O ponto de engenharia: a atenção é $O(n^2)$. Custo por token, tamanho da janela e toda a economia de contexto nascem dessa quadrática.

► Roteiro de fala

Único slide de arquitetura em que vale gastar 3 minutos — tudo depois é engenharia em cima desta caixa. Suba o diagrama de baixo para cima: o texto vira **tokens** (pedaços de palavra), os tokens viram **embeddings** (vetores) com a posição codificada (RoPE, codificação posicional rotativa). Aí entra o bloco que se repete N vezes: **atenção multi-cabeça**, soma residual e normalização, rede *feed-forward*, de novo soma e normalização. No topo, uma projeção vira **logits**, o **softmax** vira probabilidade sobre o vocabulário, e a amostragem escolhe o próximo token. Explique Q, K, V em 40 s com a analogia de busca: cada token faz uma pergunta (*query*), todos anunciam o que têm (*keys*), e a semelhança decide de quem ele copia informação (*value*); várias 'cabeças' fazem isso em paralelo. **O ponto que a plateia tem que levar:** a atenção custa $O(n^2)$ — cresce com o quadrado do tamanho da entrada. É daí que vêm o custo por token, o limite de janela e toda a indústria de 'contexto longo' que veremos no S13. (~3 min)

Por que o Transformer venceu

BLOCO 2 · LLMS

Por que o Transformer venceu

Recorrência (LSTM) é sequencial no treino. Uma palavra depois da outra — não aproveita a placa de vídeo.

Atenção é paralelizável. Todos os tokens de uma vez, na GPU. Foi isso que mudou o jogo.

Não venceu por qualidade por token — venceu por VAZÃO de treinamento. Deu para treinar modelos muito maiores em tempo viável. Escala, não elegância.

A arquitetura de hoje: decoder-only, autorregressiva, objetivo único. Um só alvo de treino: prever o próximo token.

► Roteiro de fala

Um minuto, uma ideia contraintuitiva. A LSTM — a rede recorrente que dominava sequências — processa palavra por palavra, em ordem, então o treino é sequencial e não aproveita a placa de vídeo. A atenção olha todos os tokens ao mesmo tempo, então **paraleliza**. O Transformer não ganhou por ser melhor 'por palavra'; ganhou por **vazão** (throughput): deu para treinar modelos muito maiores em tempo viável. E feche com a frase que desmistifica tudo: os modelos que vocês usam são **decoder-only**, autorregressivos, com um único objetivo de treino — prever o próximo token. Diga com todas as letras: *'não existe um módulo de raciocínio, nem um módulo de código; existe previsão de próximo token, e o raciocínio e o código emergiram dela'*. Isso reposiciona a plateia: o modelo é um previsor estatístico, não um pensador — e é por isso que precisamos de portões de verificação. (~1 min)

Escala e o que emerge dela

BLOCO 2 · LLMS

Escala e o que emerge dela

Leis de escala. Mais dados + mais parâmetros + mais processamento → menos erro, de forma previsível. Foi o que justificou o investimento.

O salto de 2020: aprendizado no contexto (in-context learning). O modelo aprende a tarefa na hora da resposta, pelos exemplos do próprio prompt, sem tocar nos pesos.

Consequência: prompt e RAG passam a funcionar. E o ajuste fino (fine-tuning) deixou de ser o caminho obrigatório.

Deslocamento recente: do processamento no TREINO para o processamento na RESPOSTA. O modelo “pensa mais” na hora — volta idêntico no AI Co-Scientist (S62).

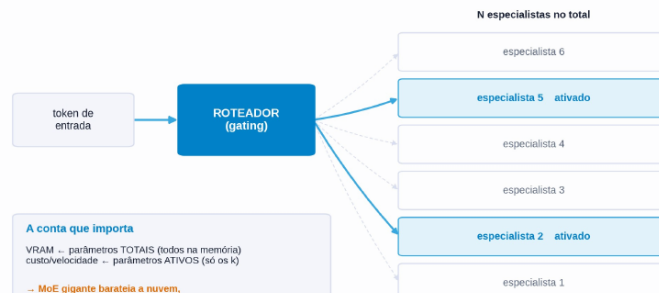
► Roteiro de fala

90 segundos sobre o que a escala trouxe de graça. As **leis de escala** são a descoberta de que o erro cai de forma previsível conforme você aumenta dados, parâmetros e capacidade de processamento — foi isso que justificou o investimento bilionário. Mas o salto conceitual de 2020 foi o **aprendizado no contexto** (in-context learning): o modelo passa a aprender a tarefa a partir dos exemplos que você põe no próprio prompt, na hora da resposta, **sem retreinar**. É esse fenômeno que faz a engenharia de prompt e o RAG funcionarem — e que tirou o *ajuste fino* de ser o caminho obrigatório. Termine marcando um ponto que volta mais tarde: a fronteira recente deslocou o processamento do **treino** para a **resposta** — modelos que ‘pensam mais’ antes de responder. No S62, o AI Co-Scientist da Google usa exatamente isso: mais processamento na hora da resposta produz hipóteses melhores. (~1,5 min)

MoE e arquiteturas esparsas

BLOCO 2 · LLMS

MoE: mistura de especialistas



MoE = Mixture of Experts (mistura de especialistas). Ex.: DeepSeek-V4-Pro tem 1,6 trilhão de parâmetros totais, mas só 49 bilhões ativos por token.

► Roteiro de fala

90 segundos, e o ponto é prático, não teórico. MoE — mistura de especialistas — é uma arquitetura onde um **roteador** escolhe, para cada token, apenas **k** especialistas de um total de **N**. Então o modelo tem duas contagens de parâmetros bem diferentes: os **totais** e os **ativos por token**. Exemplo: o DeepSeek-V4-Pro tem 1,6 trilhão total, mas só 49 bilhões ativos. Aqui está a consequência que a plateia — que hospeda os próprios modelos — precisa ouvir: a sua conta de **VRAM** (memória da placa de vídeo) é ditada pelos parâmetros **totais**, porque todos precisam estar carregados; já o custo e a velocidade dependem só dos **ativos**. Traduzindo: um MoE gigante barateia a resposta **na nuvem** e **não ajuda quem roda em casa**. E o contrário também vale — um modelo **denso** pequeno pode superar um MoE muito maior; a linha Qwen de 27 bilhões (densa) fez isso. É a deixa perfeita para o S12. (~1,5 min)

Inferência: os parâmetros que importam

BLOCO 2 · LLMS

Inferência: os parâmetros que importam

Temperatura e top-p. Controlam se a amostragem é mais criativa ou mais determinística.

Cache de chaves-valores (KV cache) e cache de prefixo. Reaproveitam contas já feitas — menos custo, menos espera.

Quantização. Reduz a precisão dos pesos (ex.: 4 bits) para caber em menos memória de vídeo.

Regra prática: código e extração estruturada pedem temperatura baixa; exploração de hipóteses pede o contrário — gancho para o S62.

```
# a MESMA chamada, dois resultados

resp = modelo.gerar(prompt,
    temperatura=0.0)
# código / extração: baixa

resp = modelo.gerar(prompt,
    temperatura=1.0)
# exploração: alta
```

► Roteiro de fala

90 segundos sobre os botões que vocês realmente vão girar. A **temperatura** (e o top-p) controla se a amostragem é mais criativa ou mais determinística. Mostre o trecho de código: a mesma chamada com temperatura 0 e com temperatura 1 dá resultados diferentes — e a **regra prática** é o que importa: para **gerar código e extrair dados estruturados** use temperatura baixa (você quer previsibilidade); para **explorar hipóteses**, alta. O **cache de chaves-valores** e o cache de prefixo reaproveitam contas e cortam custo e espera. A **quantização** reduz a precisão dos pesos para caber em menos memória — é o que permite rodar um modelo de 27 bilhões em 24 GB. Feche com o gancho: essa escolha de temperatura volta no S62, porque o AI Co-Scientist usa temperatura alta para gerar hipóteses e baixa para avaliá-las. (~1,5 min)

Panorama: modelos proprietários

BLOCO 2 · LLMS

Panorama: modelos proprietários

Modelo	Fabricante	SWE-bench Verified	Fonte da medição
GPT-5.6 Sol	OpenAI	~96%	Vals AI (independente)
Claude Fable 5	Anthropic	~95%	medição independente
Gemini 3.1 Pro	Google	agrupamento superior	—

Ressalva obrigatória: o número do fabricante e o de uma medição independente divergem, às vezes muito. Cite sempre a fonte.

SWE-bench Verified = teste de correção de bugs reais de software (%% de tarefas resolvidas). Última verificação: __ / __ / ____

► Roteiro de fala

Um minuto, e cuidado: este slide envelhece em semanas. Mostre a fronteira dos modelos fechados em **SWE-bench Verified** — um teste que mede resolução de bugs reais de software, então é o mais relevante para dev. Os números de meados de 2026 põem GPT-5.6, Claude Fable 5 e Gemini 3.1 no topo. Mas faça a **ressalva obrigatória em voz alta**: o número que o fabricante publica e o número que uma medição independente (como a Vals AI) encontra **divergem**, às vezes muito — então sempre diga a fonte. Aponte o rodapé 'última verificação': reconfira estes valores na semana da palestra; dado errado na frente de gente sênior custa credibilidade. (~1 min)

Panorama: modelos abertos para código

BLOCO 2 · LLMS

Panorama: modelos abertos para código

Modelo	Arquitetura	Contexto	Licença	Onde roda
Kimi K3	2,8T / 104B ativos	1M	pesos abertos	infra séria
DeepSeek-V4-Pro	1,6T / 49B ativos	1M	MIT	cluster
DeepSeek-V4-Flash	284B / 13B ativos	1M	MIT	servidor médio
GLM-5.2	MoE	longo	MIT	servidor
Qwen3.8-27B	denso 27,8B · multimodal	262K – 1M	Apache-2.0	24 GB VRAM
Qwen3.6-27B	denso 27B	longo	Apache-2.0	1 GPU consumo
Muse Glimmer 30B	denso · multimodal	131K	Apache-2.0	24 GB quantiz.
MiniMax M3	428B / 23B ativos	1M	comunitária ⚠	ver licença

O caso que importa para vocês: Qwen3.8-27B — denso, multimodal, Apache-2.0, roda em 24 GB e, em SWE-bench Pro, supera o Claude Opus 4.6 Max.

Alerta jurídico: “pesos abertos” ≠ “código aberto”. MIT e Apache-2.0 são livres; a licença do MiniMax é não comercial por padrão. · Última verificação: __/__/__

► Roteiro de fala

É o slide que a plateia mais vai fotografar — dê tempo (2,5 min) e estrutura. Primeiro leia a tabela em diagonal: modelos gigantes de nuvem no topo (Kimi K3, DeepSeek-V4-Pro), e desça até a linha que interessa a quem hospeda: **Qwen3.8-27B**. Agora o 'slide dentro do slide': este modelo, denso de 27 bilhões, multimodal, licença Apache-2.0 (livre até para uso comercial), **roda em 24 GB de memória de vídeo** — uma placa de estação. Contra a geração anterior, ele deu saltos de dois dígitos em tarefas de agente **sem aumentar de tamanho**. E o número que cala a sala: em **SWE-bench Pro** ele supera o Claude Opus 4.6 Max. Mas seja honesto — *'a fronteira não caiu, ficou irregular'*: ele ainda perde em outros testes. Duas ressalvas ditas de propósito: (1) esses números são do cartão do modelo, do fabricante — mas por ser aberto, qualquer um repete, o que é argumento **a favor**; (2) os pesos são o piso, não o total — o cache de chaves-valores vem por cima e pode dobrar a memória com contexto longo. E o **alerta jurídico**, para uma empresa que vende software: 'pesos abertos' não é 'código aberto'; MIT e Apache são livres, mas a licença do MiniMax é não comercial por padrão — decisão jurídica, não técnica. (~2,5 min)

Contexto longo: o número anunciado é o menos útil

BLOCO 2 · LLMS

Contexto longo: o número anunciado é o menos útil

Anúncios de 1 a 10 milhões de tokens. Vários modelos anunciam janelas gigantes — sem um teste publicado que sustente qualidade perto disso.

Degradação do contexto (“context rot”). A capacidade efetiva fica em 60–70% do anunciado; e a queda não é gradual — segura até um limiar e despenca. Uma janela de 1M pode degradar já em 50 mil.

Resultado contraintuitivo (testes da Chroma). Os modelos foram melhores com texto embaralhado do que coerente: texto coerente cria viés de recência — sobrepeça o fim e esquece o começo.

Custo. Encher 1 milhão de tokens vai de centavos a dezenas de dólares — diferença de dezenas de vezes entre modelos.

Contexto menor e curado supera contexto grande e velho. Pior em domínio regulado: quando a janela estoura, o modelo corta em silêncio — sem registro do que se perdeu.

Engenharia de IA & Desenvolvimento Orientado a Especificação

13 / 68

► Roteiro de fala

Dois minutos que fecham o bloco de LLMS e derrubam um mito de marketing. Todo fabricante anuncia janelas gigantes — 1 milhão, 10 milhões de tokens. Diga a verdade incômoda: esse número é o **menos útil** da ficha técnica. Existe um fenômeno medido, a **degradação do contexto** (context rot): a capacidade **efetiva** costuma ser 60 a 70% do anunciado, e pior, a queda não é suave — o modelo segura a qualidade até um limiar e então **despenca**; uma janela de 1 milhão pode começar a degradar já em 50 mil. Conte o resultado contraintuitivo dos testes da Chroma: os modelos foram **melhores com o texto embaralhado do que coerente**, porque texto coerente cria um viés de recência — o modelo dá peso demais ao final e esquece o começo. Toque no custo: encher 1 milhão de tokens custa de centavos a dezenas de dólares dependendo do modelo. Feche com a frase-tese do bloco: *‘contexto menor e curado supera contexto grande e velho’*. E o agravante para software de saúde, que volta no S51: quando a janela estoura, o modelo **corta em silêncio** — não fica registro do que foi perdido, e num sistema auditável isso é inaceitável. (~2 min)

A escada de abstração

BLOCO 3 · SOFTWARE

A escada de abstração

Assembly → **linguagem de alto nível**. Objeção da época: “vai gerar código lento e sem controle”. Consolidou-se com o **COMPILADOR**.

Alto nível → **frameworks**. Objeção: “esconde o que importa, vira mágica”. Consolidou-se com o **VERIFICADOR DE TIPOS** e os **TESTES**.

Frameworks → **especificação + verificação**. É o degrau de agora. A mesma objeção — “não dá para confiar” — e a mesma resposta: uma verificação nova.

O padrão nunca muda. Cada degrau sobe o nível do que você escreve — e só se firma quando ganha a sua camada de verificação automática.

► Roteiro de fala

Um minuto para desarmar o ceticismo com história. Toda subida de abstração enfrentou a mesma objeção e venceu do mesmo jeito. Do assembly para a linguagem de alto nível, diziam 'vai gerar código ruim' — e o **compilador** resolveu. Da linguagem para os frameworks, diziam 'esconde o que importa' — e o **verificador de tipos e os testes** resolveram. Agora estamos no degrau seguinte: dos frameworks para **especificar e verificar**. A objeção é idêntica ('não dá para confiar em código de IA') e a resposta será idêntica: uma nova camada de verificação — que é exatamente o que o Bloco 9 vai mostrar. Diga a frase: 'cada degrau só se firmou quando ganhou a sua verificação'. (~1 min)

O que a IA faz bem e mal, e quanto custa

BLOCO 3 · SOFTWARE

O que a IA faz bem e mal em código

FAZ BEM

- código de fronteira **bem especificado**
- traduzir entre representações
- gerar **testes a partir de critérios explícitos**
- refatoração mecânica

FAZ MAL

- decisão arquitetural com **trade-off implícito**
- corretude que depende de **contexto não escrito**
- invariantes **não declarados**
- regras que “todo mundo sabe” e ninguém anotou

A coluna da direita é toda ela **falta de especificação**. Custo típico: 50 mil a 500 mil tokens por tarefa; o português gasta ~1,5× mais tokens que o inglês.

► Roteiro de fala

Um minuto, e o fecho é o gancho da palestra inteira. A IA é ótima no que está **bem especificado**: gerar código de um problema bem descrito, traduzir entre representações, escrever testes a partir de critérios explícitos, refatorar. E é ruim exatamente onde **falta especificação**: decisões de arquitetura com compromissos implícitos, corretude que depende de contexto que ninguém escreveu, invariantes não declarados. Aponte a coluna da direita e diga a frase: *‘isto tudo é falta de especificação’* — plantando o Bloco 9. Feche com o custo, para eles terem noção de escala: uma tarefa de agente gasta de 50 mil a 500 mil tokens, e o português consome cerca de 1,5 vez mais tokens que o inglês — importa na conta. (~1 min)

De prompt para contexto

BLOCO 4 · CONTEXTO E RAG

De “prompt” para “engenharia de contexto”

O trabalho não é mais escrever uma frase esperta. É **decidir o que ocupa a janela** — o espaço de atenção é finito e caro (lembra do $O(n^2)$ e do S13).

instruções

ferramentas

histórico

documentos (RAG)

saída

A “janela de contexto”: um orçamento fixo a ser gasto com disciplina.

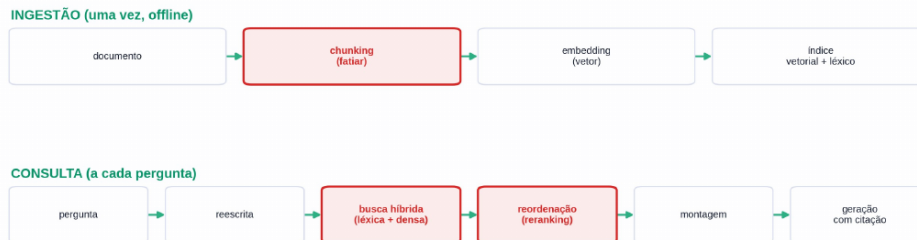
► Roteiro de fala

30 segundos para reposicionar o trabalho. Com o S13 fresco na cabeça, diga: o ofício deixou de ser 'achar o prompt mágico' e virou **engenharia de contexto** — decidir, com disciplina, o que entra na janela. Mostre a barra: a janela é um **orçamento fixo** dividido entre instruções, ferramentas, histórico, documentos recuperados (o RAG) e o espaço da saída. Cada token gasto num lugar é um token a menos em outro — e, pelo $O(n^2)$, cada token custa. (~30 s)

RAG: o diagrama

BLOCO 4 · CONTEXTO E RAG

RAG: como funciona, e onde quebra



Os três pontos de quebra: **fatiar mal**, **recuperar o trecho errado**, e a **ausência de reordenação**.

RAG raramente falha na geração — falha na recuperação, e o time gasta semanas ajustando o prompt.

RAG = Retrieval-Augmented Generation (geração aumentada por recuperação).

► Roteiro de fala

Dois minutos e meio no diagrama — é o slide técnico central do bloco. RAG significa geração aumentada por recuperação (Retrieval-Augmented Generation): em vez de o modelo 'saber' tudo, você **busca os trechos certos e os entrega no contexto**. São dois trilhos. O de cima, a **ingestão**, roda uma vez: o documento é fatiado em pedaços (chunking), cada pedaço vira um vetor (embedding) e vai para um índice. O de baixo, a **consulta**, roda a cada pergunta: reescreve a pergunta, faz busca híbrida (por palavra e por significado), **reordena** os resultados (reranking), monta o contexto e gera a resposta com citação da fonte. Agora aponte os três pontos em vermelho — é onde o RAG morre: **fatiar mal, recuperar o trecho errado, e não reordenar**. E diga a frase que a plateia vai reconhecer da própria dor: *'RAG raramente falha na geração; ele falha na recuperação, e o time gasta semanas ajustando o prompt achando que o problema é outro'*. (~2,5 min)

Chunking ingênuo destrói um critério de IRAS

BLOCO 4 · CONTEXTO E RAG

Por que fatiar mal destrói um critério de IRAS

O critério de definição de caso (ICSAC) ocupa três parágrafos. Critério clínico + critério laboratorial + janela temporal — os três precisam ser lidos JUNTOS.

Em pedaços de 512 tokens, o laboratorial se separa da janela. A recuperação devolve metade da regra; o modelo completa o resto sozinho — e notifica um FALSO POSITIVO.

A correção: fatiar pela unidade lógica, não por tamanho fixo. Chunking semântico pela regra inteira + metadados por tipo de infecção + expansão de janela ao redor do trecho.

► Roteiro de fala

90 segundos com um exemplo do domínio deles — é o que faz o conceito grudar. Pegue um critério de definição de caso da vigilância — a sigla ICSAC, os critérios que definem quando uma infecção conta como caso. O enunciado tem três parágrafos: o **critério clínico**, o **critério laboratorial** e a **janela temporal**. Os três só fazem sentido lidos juntos. Se você fatia o documento em pedaços de 512 tokens — o padrão preguiçoso — o critério laboratorial se separa da janela temporal. A busca devolve metade da regra, o modelo 'completa' o resto por conta própria, e o resultado é uma **notificação falsa** — num sistema de vigilância, um custo real. A correção é fatiar pela **unidade lógica** (a regra inteira num pedaço só), pôr metadados por tipo de infecção e expandir a janela ao redor do trecho recuperado. (~1,5 min)

Saída estruturada como contrato

BLOCO 4 · CONTEXTO E RAG

Saída estruturada como contrato

```
{
  "critério": "ICSAC-corrente-sanguinea",
  "atendido": true,
  "evidencia_textual":
    "hemocultura + para S. aureus em 12/03",
  "data_referencia": "2026-03-12",
  "confiança": 0.91
}
```

Um esquema (schema) por critério. Booleano do resultado, o trecho literal que o justifica, a data de referência e a confiança.

Todo booleano vem com a evidência citada. Sem o trecho literal não há revisão humana viável nem trilha de auditoria.

Não peça parecer. Peça formulário preenchido. "Formulário com evidência citada", não "texto livre opinando".

► Roteiro de fala

Dois minutos — e é uma das ideias mais acionáveis da palestra. Mostre o esquema (schema) na esquerda: para cada critério de IRAS extraído do texto livre, o modelo devolve quatro campos — se o critério foi **atendido** (booleano), o **trecho literal** do prontuário que justifica, a **data de referência** e um grau de **confiança**. O ponto central: **todo booleano vem acompanhado da evidência textual que o sustenta**. Sem isso, não existe revisão humana viável — o infectologista não tem como conferir — nem trilha de auditoria. Diga a frase que resume tudo: *'não peçam um parecer ao modelo; peçam um formulário preenchido, com a evidência citada'*. Isso transforma uma saída opaca num contrato verificável. (~2 min)

Avaliação e guardrails

BLOCO 4 · CONTEXTO E RAG

Avaliação e proteções (guardrails)

Eixo	O que medir	Por quê
Estratificação	sensibilidade e especificidade POR tipo de infecção	o custo do erro muda por infecção
Conjunto dourado	casos revisados por infectologista, versionado	verdade de referência estável
Gatilho de CI	roda a cada mudança de prompt, MODELO ou base	trocar versão = mudança controlada
Asserção	um teste com assert sobre o conjunto dourado	o portão é código, não opinião

Acurácia agregada é inútil aqui: falso negativo e falso positivo têm custos assimétricos e diferentes por infecção. Guardrails: menor privilégio, anonimizar antes de sair, e cuidado com injeção vinda do próprio prontuário (LGPD).

► Roteiro de fala

90 segundos — e este slide é a recomendação nº 1 do fechamento, então dê peso. A primeira coisa: acurácia agregada é **inútil** em IRAS, porque um falso negativo (deixar passar uma infecção) e um falso positivo (notificar à toa) têm custos **assimétricos** e diferentes por tipo de infecção. Então meça pela matriz: **sensibilidade e especificidade estratificadas** por infecção; um **conjunto dourado** de casos revisados por infectologista, versionado; um **gatilho de integração contínua** que roda esse conjunto a cada mudança de prompt, de modelo ou de base; e uma **asserção** — um teste com *assert* — para que o portão seja código e não opinião. Feche com a frase forte: *'trocar a versão do modelo sem esse conjunto rodando é mudança não controlada em software de saúde'*. E cite as proteções: menor privilégio, anonimizar antes de qualquer chamada externa, e o risco de injeção de comandos vinda do próprio texto do prontuário — tudo sob a LGPD. (~1,5 min)

Definição operacional

BLOCO 5 · AGENTES

Definição operacional (sem misticismo)

Um agente é um modelo em laço, com ferramentas, sobre um ambiente, com critério de parada.

O que NÃO é agente: fluxo de caminho fixo com o modelo nos nós. Isso é um workflow — e quase sempre é a escolha certa.



Eixo da autonomia: da esquerda para a direita, mais poder e mais risco. Autonomia é custo, não virtude — escolha o mínimo que resolve.

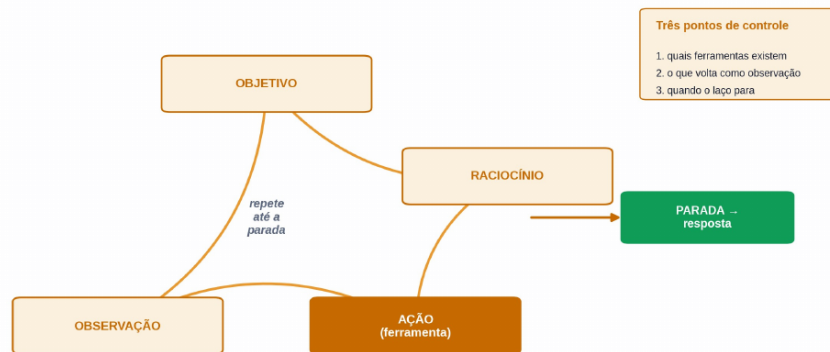
► Roteiro de fala

90 segundos para tirar o misticismo da palavra 'agente'. Leia a definição operacional, devagar: **um modelo em laço, com ferramentas, sobre um ambiente, com critério de parada**. Só isso. E diga o que **não** é agente: um fluxo de caminho fixo com o modelo em alguns nós — isso é um *workflow*, e na maioria dos casos é a escolha **certa**, mais simples e mais depurável. Mostre o eixo da autonomia: determinístico, roteado por modelo, agente com ferramentas, autônomo — e martele a frase: *'autonomia é custo e risco, não virtude; use o mínimo que resolve o problema'*. Isso prepara o S28 (por que agentes falham) e o S50 (como os gates domam isso). (~1,5 min)

O laço agêntico

BLOCO 5 · AGENTES

O laço agêntico



Três pontos de controle

1. quais ferramentas existem
2. o que volta como observação
3. quando o laço para

Um agente = um modelo em laço, com ferramentas, sobre um ambiente, com critério de parada.

► Roteiro de fala

Dois minutos no ciclo — é o coração do bloco. Percorra o laço: parte de um **objetivo**, o modelo **raciocina** sobre o que fazer, executa uma **ação** (chama uma ferramenta), recebe uma **observação** do ambiente, e **repete** até bater o critério de parada — aí devolve a resposta. É isso, e é só isso. Agora aponte os **três pontos de controle** — porque é onde você, engenheiro, realmente atua: **quais ferramentas existem** (o que o agente pode fazer), **o que volta como observação** (o que ele enxerga) e **quando o laço para** (senão ele roda para sempre ou gasta dinheiro sem controle). Guarde: quase todo problema de agente em produção é um desses três mal definidos. (~2 min)

Tool use sem framework

BLOCO 5 · AGENTES

“Tool use”: usar ferramentas sem framework nenhum

```
# 1. você DESCREVE a ferramenta (isto é prompt!)
ferramenta = {
    "nome": "consultar_microbiologia",
    "parametros": {"paciente_id": "str",
                  "janela_dias": "int"}
}
# 2. o modelo responde pedindo a chamada (tool_use)
# 3. VOCÊ executa e devolve o resultado (tool_result)
# 4. repete num while explícito até a resposta
```

O laço agêntico tem trinta linhas. Tudo que vem depois — os frameworks do Bloco 7 — é conveniência, não mágica.

A descrição da ferramenta É prompt. Descrição ruim de ferramenta causa mais falha do que prompt de sistema ruim.

► Roteiro de fala

Três minutos, e é o slide que dá confiança técnica à plateia sênior. Mostre que 'usar ferramentas' (tool use) não tem mágica nenhuma. Quatro passos: (1) você **descreve** a ferramenta num esquema — nome e parâmetros — por exemplo *consultar_microbiologia(paciente_id, janela_dias)*; (2) o modelo, em vez de responder texto, devolve um pedido de chamada (tool_use); (3) **você** executa a função de verdade e devolve o resultado (tool_result); (4) isso roda num *while* explícito até o modelo dar a resposta final. Diga a frase que desarma o hype: *'o laço agêntico tem trinta linhas; tudo que vem depois é conveniência'*. E o insight prático que eles vão usar na segunda-feira: **a descrição da ferramenta é prompt** — uma descrição ruim de ferramenta causa mais falha do que um prompt de sistema ruim. (~3 min)

Componentes

BLOCO 5 · AGENTES

Componentes de um agente

Memória curta × longa. Curta = a conversa atual; longa = o que persiste entre execuções.

Planejamento. Quebrar o objetivo em passos antes de agir.

Reflexão. Criticar o próprio resultado e tentar de novo.

Memória longa em arquivo versionado costuma superar a vetorial em regras estáveis. É auditável e comparável em diff — gancho direto para o formato OKF do S31.

► Roteiro de fala

30 segundos, rápido. Um agente combina: **memória curta** (a conversa atual) e **longa** (o que persiste entre execuções), **planejamento** (quebrar em passos) e **reflexão** (criticar o próprio trabalho). Deixe um gancho para depois: para **regras estáveis** — como os critérios de IRAS — a memória longa em **arquivo versionado** costuma superar a memória vetorial, porque é auditável e você vê a diferença num diff. Isso volta no formato OKF, no S31. (~30 s)

Os seis padrões de composição

BLOCO 5 · AGENTES

Os seis padrões de composição



Comece pelo mais simples que resolve. Os mais subtilizados e úteis aqui: VOTAÇÃO e AVALIADOR-OTIMIZADOR.

► Roteiro de fala

Três minutos — é o catálogo que a plateia vai querer fotografar. São seis maneiras de compor agentes, da mais simples à mais arriscada: **encadeamento** (a saída de um vira a entrada do próximo), **roteamento** (classifica e manda ao especialista certo), **paralelização** (secciona o trabalho, ou faz vários resolverem e **vota**), **orquestrador-trabalhadores** (um coordena, vários executam isolados), **avaliador-otimizador** (um gera, outro critica e manda refazer) e o **agente autônomo** (decide os próprios passos — mais poder, mais risco). Diga a regra de ouro: *'comecem pelo mais simples que resolve'*. E destaque os dois **mais subtilizados e mais úteis para reduzir falso positivo aqui**: a **votação** e o **avaliador-otimizador** — os dois destacados. Guarde-os: eles reaparecem no AI Co-Scientist, no S62. (~3 min)

O mesmo caso de três formas

BLOCO 5 · AGENTES

O mesmo caso resolvido de três formas

Caso: agente de vigilância de IRAS — varre microbiologia, prescrição e evolução; aplica os critérios; propõe a notificação; e PARA em revisão humana.

Encadeado

passos fixos em sequência; simples, mas rígido

Orquestrador-trabalhadores

um trabalhador por tipo de infecção, contexto isolado

Avaliador-otimizador

um crítico contesta contra o critério normativo: menos falso positivo, ~2× tokens

► Roteiro de fala

Dois minutos, e é onde a teoria vira decisão de arquitetura. Fixe um caso concreto: um **agente de vigilância de IRAS** que varre microbiologia, prescrição e evolução clínica, aplica os critérios de caso, propõe a notificação e **para em revisão humana** (nunca notifica sozinho). Agora resolva-o de três jeitos. **Encadeado**: passos fixos em sequência — simples, mas rígido. **Orquestrador-trabalhadores**: um trabalhador por tipo de infecção, cada um com contexto isolado — escala e evita contaminação. **Avaliador-otimizador**: um agente crítico contesta cada proposta contra o critério normativo — isso **reduz falso positivo**, ao custo de cerca do dobro de tokens. A lição: a arquitetura é uma escolha de compromisso, e você a faz de olho no custo do erro. (~2 min)

Multiagente: quando compensa

BLOCO 5 · AGENTES

Multiagente: quando compensa (e quando não)

Só compensa com subtarefas paralelizáveis E contextos isolados. Se as partes não são independentes, o custo explode sem ganho.

Em fluxo linear: 4 a 15× mais tokens para reimplementar um “if”. Vários agentes conversando para decidir o que um condicional resolveria.

A passagem de contexto entre agentes é “lossy”. Cada repasse perde informação — como fotocópia de fotocópia.

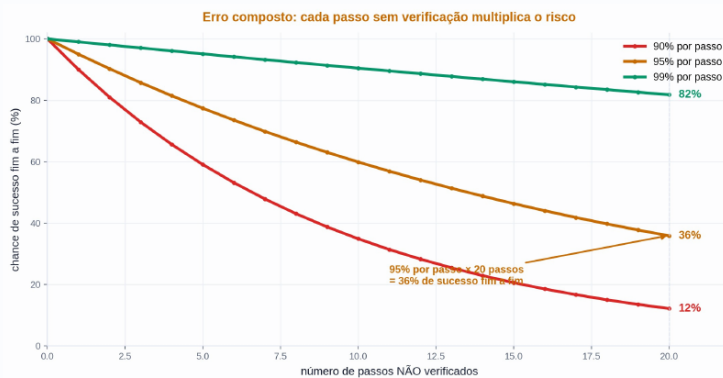
► Roteiro de fala

30 segundos de contrapeso, importante. Depois de mostrar o poder do multiagente, segure o entusiasmo: ele **só compensa** quando há subtarefas de fato paralelizáveis **e** contextos que podem ficar isolados. Num fluxo linear, montar vários agentes custa de 4 a 15 vezes mais tokens para **reimplementar o que um simples 'if' resolveria**. E lembre: a passagem de contexto entre agentes é *lossy* — perde informação a cada repasse, como fotocópia de fotocópia. A moral: multiagente é ferramenta cara; use quando o problema pede, não por elegância. (~30 s)

Por que agentes falham em produção

BLOCO 5 · AGENTES

Por que agentes falham em produção



Erro composto. 95% de acerto por passo, 20 passos → só 36% de sucesso fim a fim.

Modos de falha: laço infinito, ferramenta alucinada, contexto contaminado, custo imprevisível — e o pior, a FALHA SILENCIOSA.

A saída não é “prompt melhor”. É reduzir o número de passos NÃO verificados. Guardem esta frase para o Bloco 9.

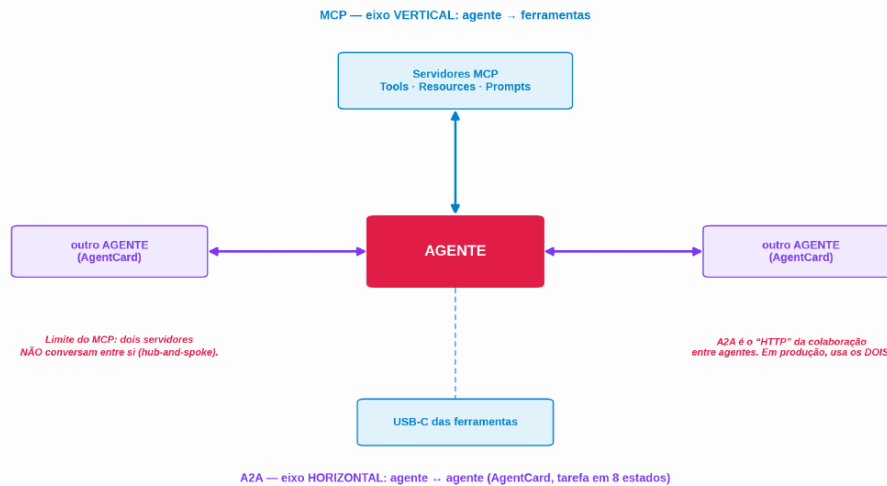
► Roteiro de fala

90 segundos, e este é o slide que arma o núcleo da palestra (o Bloco 9). A causa raiz é matemática: o **erro composto**. Mostre a curva — se cada passo acerta 95% das vezes, vinte passos em sequência dão só **36% de sucesso** fim a fim, porque as probabilidades se multiplicam. Aponte as três linhas (90, 95, 99% por passo) para mostrar como a confiabilidade por passo é decisiva. Liste os modos de falha — laço infinito, ferramenta alucinada, contexto contaminado, custo imprevisível — e destaque o mais perigoso em software de saúde: a **falha silenciosa**, quando o agente erra e ninguém percebe. E então a conclusão que vale a palestra inteira: *'confiabilidade não vem de um prompt melhor; vem de reduzir o número de passos não verificados'* — que é exatamente o que os gates do SDD fazem. (~1,5 min)

O mapa: vertical × horizontal

BLOCO 6 · PROTOCOLOS

O mapa: vertical (MCP) × horizontal (A2A)



► Roteiro de fala

Dois minutos, e a analogia final é o que fica. Dois protocolos abertos organizam o ecossistema. O **MCP** (Protocolo de Contexto de Modelo, da Anthropic) é o eixo **vertical**: liga o agente às suas **ferramentas** — servidores que expõem Tools, Resources e Prompts. O limite dele: é hub-and-spoke, ou seja, **dois servidores MCP não conversam entre si**. O **A2A** (Agente-para-Agente, do Google) é o eixo **horizontal**: liga um agente a **outros agentes**, cada um publicando um cartão de capacidades (AgentCard) e trocando tarefas com estados bem definidos. Feche com a analogia que a plateia vai levar: 'o MCP é o USB-C das ferramentas; o A2A é o HTTP da colaboração entre agentes — e em produção você usa os dois'. (~2 min)

A2Family e alternativas

BLOCO 6 · PROTOCOLOS

A família A2A e as alternativas

Extensões oficiais do A2A. Secure Passport (identidade), Timestamp, Traceability (rastreadibilidade), Agent Gateway.

A família: AP2, UCP, A2UI. Pagamentos iniciados por agente, comércio com consentimento criptográfico, e interface.

Alternativas: ACP (IBM) e ANP. ACP herda a tradição FIPA-ACL com performativas tipadas; ANP aposta em identidade descentralizada.

A crítica que dá valor ao slide: nenhum protocolo expressa GOVERNANÇA. Registram quem chamou quem — não sob qual política nem com que base legal. Em saúde, essa camada sobra para VOCÊS.

► Roteiro de fala

90 segundos, e o remate é o gancho comercial. Em volta do A2A cresceu uma família: extensões oficiais (passaporte seguro, carimbo de tempo, rastreabilidade, portão de agentes) e protocolos irmãos — AP2 para pagamentos iniciados por agente, UCP para comércio com consentimento criptográfico, A2UI para interface. Há alternativas: o ACP da IBM, que herda a tradição acadêmica FIPA-ACL com mensagens tipadas (propor, aceitar, recusar), e o ANP, com identidade descentralizada. Mas o ponto que valoriza o slide é a **crítica**: pesquisa recente mostra que **nenhum desses protocolos expressa governança** — eles registram quem chamou quem, mas não sob qual política nem com que base legal. Em software de saúde, essa camada de governança **sobra para a arquitetura de vocês**. Isso volta no S51 e no S65. (~1,5 min)

OKF: Open Knowledge Format

BLOCO 6 · PROTOCOLOS

OKF: um formato aberto de conhecimento

① Um pacote (bundle) é um diretório de arquivos markdown. Nada de banco de dados, nada de servidor.

② Cada arquivo é um conceito; o caminho do arquivo é o identificador.

③ Cada arquivo abre com frontmatter YAML — único campo obrigatório: `type`.

④ Arquivos se referenciam por links markdown comuns. O que transforma o diretório num GRAFO, não numa lista.

⑤ Dois nomes reservados: `index.md` (listagem) e `log.md` (histórico).

Para vocês, direto:

um pacote OKF com as definições de caso de IRAS, as fórmulas dos indicadores e os runbooks da CCIH.

Legível por agente, revisável por infectologista, com `log.md` de histórico e “git blame” de autoria —

metade do que uma auditoria já pede.

Ressalva: é v0.1 — padroniza o contêiner, não a semântica clínica.

► Roteiro de fala

Dois minutos num formato simples que resolve um problema real de vocês. Enquanto os protocolos definem **comunicação**, o OKF (Formato Aberto de Conhecimento, do Google Cloud) define **conhecimento** — e a especificação inteira cabe em cinco regras. Leia as cinco na tela: um pacote é um diretório de arquivos markdown (sem banco, sem servidor); cada arquivo é um conceito e o caminho é o identificador; cada arquivo abre com um cabeçalho YAML cujo único campo obrigatório é o `type`; os arquivos se referenciam por links markdown, o que faz do diretório um **grafo**; e há dois nomes reservados, `index.md` e `log.md`. Agora a aplicação direta, e diga com estas palavras: um pacote OKF com as **definições de caso de IRAS**, as **fórmulas dos indicadores** (densidade de incidência, taxa de uso de dispositivo) e os **runbooks da comissão de controle de infecção** é exatamente o conhecimento que hoje vive em PDF, planilha e na cabeça das pessoas. Em OKF ele fica legível por agente, revisável por infectologista, com histórico em `log.md` e autoria por `git blame` — que é metade do que uma auditoria pede. Ressalva honesta: é versão 0.1, padroniza o recipiente, não a semântica clínica — se adotarem, vocês definem o vocabulário. (~2 min)

Os outros formatos em markdown

BLOCO 6 · PROTOCOLOS

Os outros formatos em markdown

AGENTS.md. Contexto do projeto para agentes de código: comandos de build e teste, convenções, decisões. Markdown puro.

DESIGN.md. Identidade visual: tokens de design em YAML mais o racional em prosa.

Agent Skills. Capacidade empacotada como uma pasta com um arquivo markdown, com registros públicos.

O padrão por trás dos três: markdown versionado no repositório. A única coisa que humano e máquina leem, que o git versiona e que o revisor audita num diff.

► Roteiro de fala

Um minuto para revelar um padrão maior. Além do OKF, três formatos viraram convenção: o **AGENTS.md**, que dá ao agente o contexto do projeto (como buildar, como testar, as convenções); o **DESIGN.md**, para identidade visual; e as **Agent Skills**, capacidades empacotadas como pasta com um markdown. O ponto não são os três — é o **padrão por trás deles**: o setor convergiu, sem combinar, para **markdown versionado no repositório** como o formato de contexto para agente. E não foi por elegância: é a única coisa que humano e máquina leem, que o git versiona e que o revisor consegue auditar num diff. Guarde essa frase — ela é a ponte para o SDD. (~1 min)

A ponte para o SDD

BLOCO 6 · PROTOCOLOS

A ponte para o SDD

Especificação (SDD)	AGENTS.md	Pacote OKF
o QUE o sistema deve fazer	COMO se trabalha no repositório	o DOMÍNIO que ele modela

São a MESMA ideia em escopos diferentes: conhecimento formalizado em markdown versionado, consumido por agente e revisável por humano. Os três convivem no mesmo repositório — e é isso que constrói a rastreabilidade (guardem para o S49).

► Roteiro de fala

Um minuto que costura o bloco todo e abre o núcleo da palestra. Mostre os três lado a lado e diga que são a **mesma ideia** em escopos diferentes: a **especificação** do SDD descreve *o que* o sistema deve fazer; o **AGENTS.md** descreve *como* se trabalha no repositório; o **pacote OKF** descreve o *domínio* que ele modela. Os três são conhecimento formalizado em markdown versionado, consumido por agente e revisável por humano. Termine com a frase que reaparece no S49: *'os três convivem no mesmo repositório, e é isso que constrói a rastreabilidade'*. É a deixa perfeita para o intervalo e para o mergulho no SDD. (~1 min)

Intervalo

INTERVALO

5 minutos

Na volta: frameworks, ambientes agênticos e o núcleo — o SDD.

► Roteiro de fala

Intervalo de 5 minutos. Anuncie o horário exato de retorno em voz alta e escreva no canto do quadro se possível. Avise que a volta começa pelos frameworks e caminha para o coração da palestra, o SDD. Use o intervalo para conferir se a demonstração gravada (S61) está pronta para tocar.

Critério de leitura

BLOCO 7 · FRAMEWORKS

Critério de leitura para os cinco

A pergunta única para cada framework: **o que ele controla por mim — o laço, o estado, ou nenhum dos dois?**

O mesmo agente do S26 aparece nos cinco. Para comparar o custo da abstração — não a sintaxe.

Frameworks não são mágica; são conveniência sobre o laço de 30 linhas do S23. A pergunta certa não é “qual é mais bonito”, é “o que ele assume por você e o que ele esconde”.

► Roteiro de fala

30 segundos para dar à plateia uma lente única. Em vez de decorar a interface de programação (a API) de cada framework, façam a mesma pergunta a todos: **o que este framework controla por mim — o laço, o estado, ou nenhum dos dois?** Avise que você vai usar o **mesmo agente do S26** nos cinco, de propósito, para comparar o custo da abstração e não a sintaxe. E lembre o S23: no fundo, todos embrulham o mesmo laço de trinta linhas — a pergunta é o que cada um assume por você e o que esconde. (~30 s)

OpenAI Agents SDK

BLOCO 7 · FRAMEWORKS

OpenAI Agents SDK

Peças: Agent, handoffs, guardrails, sessions. Repasse entre agentes (handoff), proteções e sessões, com rastreamento embutido.

Modelo de controle: o SDK controla o laço; o estado vive na sessão.

Rastreamento (tracing) de primeira linha — porém acoplado ao provedor. Ótima observabilidade, ao custo de amarrar você à OpenAI.

```
# triagem -> especialista por infecção
triagem = Agent(
    name="triagem",
    handoffs=[uti_agent,
               cirurgia_agent])
# o SDK cuida do laço e do repasse
```

► Roteiro de fala

Dois minutos. O SDK de Agentes da OpenAI dá quatro peças: **Agent**, **handoffs** (o repasse de uma conversa para um agente especialista), **guardrails** (proteções) e **sessions** (sessões), com rastreamento embutido. Mostre o trecho: uma triagem que faz *handoff* para o especialista por tipo de infecção. O modelo de controle: **o SDK controla o laço; o estado vive na sessão.** O rastreamento é excelente — mas **acoplado ao provedor**, então é ótima observabilidade ao custo de amarrar você à OpenAI. Boa porta de entrada; pense na dependência. (~2 min)

Anthropic Claude Agent SDK

BLOCO 7 · FRAMEWORKS

Anthropic Claude Agent SDK

O laço do Claude Code exposto como biblioteca. Busca agêntica no sistema de arquivos SEM indexação prévia, subagentes com contexto isolado, hooks.

Sistema de permissões + MCP nativo + modo headless. Controle fino do que o agente pode fazer, e execução sem interface para rodar em CI.

Diferencial: projetado em torno de CÓDIGO e ARQUIVOS, não de conversa. É o mais relevante para o Bloco 9 (o SDD roda sobre arquivos versionados).

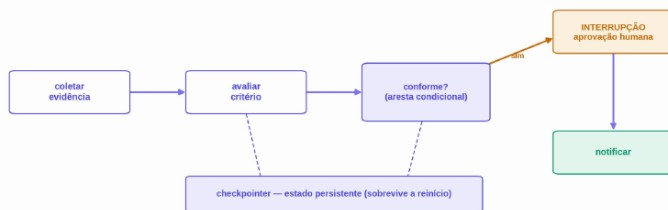
► Roteiro de fala

Dois minutos. O SDK de Agente da Anthropic é o próprio laço do Claude Code exposto como biblioteca: **busca agêntica no sistema de arquivos sem indexação prévia** (ele explora o repositório na hora), **subagentes** com contexto isolado, **hooks** (ganchos para rodar verificações dentro do laço), um **sistema de permissões**, suporte nativo ao MCP e um **modo headless** para rodar em integração contínua. O diferencial, e é o que importa para o Bloco 9: ele é projetado em torno de **código e arquivos**, não de conversa — que é exatamente o terreno do SDD. (~2 min)

LangChain e LangGraph

BLOCO 7 · FRAMEWORKS

LangChain e LangGraph



LangChain compõe componentes;
LangGraph é máquina de estados.

Estado durável (checkpoint) + parada
para aprovação (interrupt). Nativos — e
decisivos aqui.

Para IRAS: a notificação NÃO sai sem
aprovação, e o processo sobrevive a
reinício.

► Roteiro de fala

Dois minutos, e é o framework que mais importa para software regulado. Separe as duas coisas: o **LangChain** é uma biblioteca para compor componentes; o **LangGraph** é uma **máquina de estados**. Mostre o grafo: nós (coletar evidência, avaliar critério), **arestas condicionais** (conforme? sim/não), um **checkpoint** que torna o estado **durável** — sobrevive a reinício — e uma **interrupção** para **aprovação humana** antes de agir. Diga por que isso é decisivo em IRAS: a **notificação não sai sem a aprovação** do infectologista, e o processo precisa **sobreviver a um reinício** do servidor sem perder onde estava. Durabilidade e humano-no-laço nativos: é o argumento que vende o LangGraph para produção regulada. (~2 min)

CrewAI

BLOCO 7 · FRAMEWORKS

CrewAI

Modelo mental: organização, não grafo. Agent com papel, objetivo e história; Task com saída esperada; Crew reunindo; Process sequencial ou hierárquico.

Onde brilha: protótipo e demonstração. A menor curva de entrada; decompor por papel força o time a escrever o que cada agente faz — meio caminho para uma spec.

Onde dói: a metáfora de papéis ESCONDE o laço. Quando falha, você depura uma abstração, não uma sequência de chamadas. Estado e durabilidade fracos vs. LangGraph.

Veredito: ótimo para levantar um piloto numa tarde. Pense duas vezes antes de pôr em produção regulada. (É o framework do nosso LangNet — Bloco 11.)

► Roteiro de fala

Dois minutos, com honestidade — é o framework que o nosso LangNet usa. O modelo mental do CrewAI é uma **organização**, não um grafo: cada *Agent* tem papel, objetivo e história; cada *Task* tem uma saída esperada; a *Crew* reúne os agentes; o *Process* é sequencial ou hierárquico. **Onde brilha:** protótipo e demonstração — é a menor curva de entrada do grupo, e decompor por papel **força o time a escrever o que cada agente faz**, o que sem querer é meio caminho para uma especificação. **Onde dói, e diga:** a metáfora de papéis **esconde o laço** — quando falha, você depura uma abstração, não uma sequência de chamadas; a 'história' do agente é prompt disfarçado; e estado e durabilidade são fracos comparados ao LangGraph. **Veredito:** ótimo para levantar um piloto numa tarde e mostrar à direção; pense duas vezes antes de produção regulada. E o que aprendemos apanhando dele é o que o Bloco 11 mostra resolvido com portão determinístico. (~2 min)

AutoGen / AG2

BLOCO 7 · FRAMEWORKS

AutoGen / AG2

Modelo mental: conversa entre agentes. ConversableAgent como unidade; GroupChat com um gerente que escolhe quem fala; executor de código embutido.

Humano-no-laço nativo e granular. O modo de intervenção é configurável por agente (sempre, nunca, só ao terminar) — direto no gate de notificação de vocês.

Onde brilha: exploração e pesquisa, caminho não conhecido de antemão. É o parente mais próximo do AI Co-Scientist (S62).

Onde dói: comportamento emergente — a conversa pode divergir e gastar tokens sem convergir. E cuidado com a linhagem: Microsoft × fork comunitário AG2, exemplos incompatíveis.

► Roteiro de fala

Dois minutos — o mais interessante e o mais arriscado. O modelo mental do AutoGen é uma **conversa entre agentes**: a unidade é o *ConversableAgent*, um *GroupChat* tem um gerente que decide quem fala em seguida, há um ponto de entrada humano e um **executor de código** embutido — o agente escreve, roda e vê o resultado. O grande trunfo para vocês: o **humano-no-laço é nativo e granular** — dá para configurar por agente quando ele pede intervenção (sempre, nunca, só ao terminar), o que encaixa direto no portão de aprovação da notificação. **Onde brilha:** exploração e pesquisa, quando o caminho não é conhecido — é o parente mais próximo do AI Co-Scientist do S62. **Onde dói:** comportamento emergente — a conversa pode divergir, repetir e gastar tokens sem convergir. E uma nota prática: o projeto se dividiu entre a linha da Microsoft e o fork comunitário AG2, então confira a linhagem antes de copiar exemplo. (~2 min)

Tabela comparativa e a opinião contrária

BLOCO 7 · FRAMEWORKS

Comparativo e a opinião contrária

Framework	Controle do laço	Estado	Humano-no-laço	Caso ideal
OpenAI SDK	controla o laço	sessão	guardrails	produtos OpenAI
Claude Agent SDK	controla o laço	arquivos	hooks + permissões	código / arquivos
LangGraph	máquina de estados	durável	nativo (interrupt)	produção regulada
CrewAI	esconde o laço	fraco	limitado	protótipo rápido
AutoGen / AG2	conversa	fraco	granular	exploração / pesquisa
laço próprio	você controla	você faz	você faz	70% dos casos

A opinião contrária: para uns 70% dos casos, o laço direto é mais simples de depurar. Framework se justifica por durabilidade, observabilidade e humano-no-laço — não por elegância. "Não adotem framework antes de ter o eval do S20."

► Roteiro de fala

90 segundos — leia em voz alta só a última coluna da tabela. Passe o olho pelas linhas, mas leia mesmo só o **caso ideal** de cada um: OpenAI SDK para quem vive no ecossistema deles; Claude Agent SDK para código e arquivos; **LangGraph para produção regulada** (durável, humano-no-laço nativo); CrewAI para protótipo rápido; AutoGen para exploração; e o **laço próprio** para a maioria dos casos. Aí venha com a **opinião contrária**, que dá honestidade à palestra: para uns 70% dos casos, o laço direto é mais fácil de depurar; framework se justifica por **durabilidade, observabilidade e humano-no-laço**, não por elegância. E feche com a regra: *'não adotem framework antes de ter o conjunto de avaliação do S20'.* (~1,5 min)

Mudança de unidade de trabalho

BLOCO 8 · AMBIENTES

Mudou a unidade de trabalho

Do autocompletar de uma linha para a tarefa sobre o repositório inteiro.

A revisão passa a ser de um **diff inteiro**, não de uma **sugestão de linha**. Isso muda o code review do time — e é uma mudança de processo, não de ferramenta.

► Roteiro de fala

Um minuto de enquadramento. Os ambientes agênticos de código mudaram a **unidade de trabalho**: saímos do autocompletar de uma linha para a **tarefa sobre o repositório inteiro** — 'implemente este requisito', não 'complete esta linha'. A consequência prática, e é de processo: a revisão do time passa a ser de um **diff inteiro** gerado por IA, não de uma sugestãozinha. Isso muda como o code review funciona — e é bom avisar o time antes, não depois. (~1 min)

Claude Code

BLOCO 8 · AMBIENTES

Claude Code

CLI com busca agêntica sem indexação prévia. Explora o repositório na hora, em vez de manter um índice que envelhece.

CLAUDE.md, subagentes, skills, MCP, modo headless para CI. O contexto do projeto e as capacidades ficam versionados no repositório.

Hooks são o mecanismo de PORTÃO. Lint, teste e política rodam DENTRO do laço — não depois. É o gate do Bloco 9 acontecendo na prática.

► Roteiro de fala

Dois minutos. O Claude Code é uma ferramenta de linha de comando com **busca agêntica sem indexação prévia** — ele explora o repositório na hora, então nunca trabalha com um índice desatualizado. Traz o **CLAUDE.md** (o contexto do projeto versionado), subagentes, skills, MCP e um **modo headless** para rodar em integração contínua. Mas o que interessa para a tese da palestra são os **hooks**: eles são o **mecanismo de portão**. Lint, teste e verificação de política rodam **dentro do laço**, não depois — ou seja, é o *gate* do Bloco 9 acontecendo já aqui, na ferramenta. (~2 min)

Cursor e o panorama

BLOCO 8 · AMBIENTES

Cursor e o panorama

Cursor: IDE com indexação prévia, Composer/Agent, regras de projeto. Indexa antes (rápido, mas o índice pode ficar defasado).

O trade-off de fundo: indexação prévia (rápida, risco de índice velho) × exploração sob demanda (lenta, sempre atual).

O resto do panorama, numa linha: Copilot, Windsurf, Aider, Codex, Devin — variações do mesmo tema.

► Roteiro de fala

Um minuto para fechar o bloco. O **Cursor** é um ambiente de desenvolvimento (IDE) com **indexação prévia**: ele monta um índice do código antes, o que deixa a busca rápida — ao risco de o índice ficar defasado. Esse é o trade-off de fundo do bloco: **indexar antes** (rápido, mas pode envelhecer) versus **explorar sob demanda** como o Claude Code (mais lento, sempre atual). E o resto do panorama numa linha só — Copilot, Windsurf, Aider, Codex, Devin — são variações do mesmo tema. (~1 min)

O problema (vibe coding)

BLOCO 9 · SDD

O problema: “vibe coding”

Programar “no feeling” funciona no protótipo e colapsa no sistema. Enquanto é pequeno, tudo bem. Quando cresce, ninguém mais segura na cabeça.

O sintoma: o código existe, funciona — e ninguém sabe qual requisito ele atende, nem se ainda atende.

Em domínio regulado, isso tem outro nome: NÃO CONFORMIDADE. Não é dívida técnica; é risco regulatório.

► Roteiro de fala

90 segundos para nomear a dor. O 'vibe coding' — programar no feeling, pedindo pedaços à IA e colando — funciona lindamente no protótipo e **colapsa no sistema**. O sintoma é traiçoeiro: o código **existe e funciona**, mas ninguém sabe **qual requisito ele atende** nem se ainda atende. Num sistema comum isso é dívida técnica. Em **domínio regulado** — software de saúde — isso tem outro nome, e diga com todas as letras: **não conformidade**. É a transição perfeita para mostrar a alternativa. (~1,5 min)

A inversão

BLOCO 9 · SDD

A inversão

(a) **O jeito de hoje:** requisito informal → código → documentação que já nasce desatualizada.

(b) **O SDD:** especificação PRIMÁRIA → plano → tarefas → código DERIVADO.

A spec é o que se versiona, revisa e mantém. O código é o que se regenera. “É a relação entre código-fonte e binário — ninguém revisa o binário.”

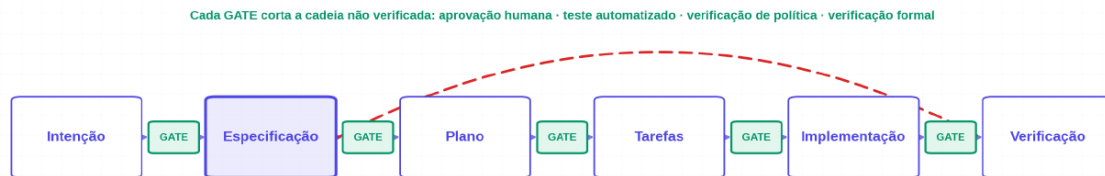
► Roteiro de fala

Dois minutos para a ideia mais importante do bloco. Contraste os dois fluxos. Hoje (a): um requisito informal vira código, e a documentação nasce atrás, desatualizada. No SDD (b): a **especificação é primária** — dela derivam o plano, as tarefas e o **código**. A inversão é essa: **a spec é o que se versiona, revisa e mantém; o código é o que se regenera**. E a analogia que faz a ficha cair na plateia de dev: *‘é a mesma relação entre código-fonte e binário — ninguém revisa o binário; você revisa a fonte e recompila’*. No SDD, a spec é a fonte e o código é o binário. (~2 min)

O ciclo SDD

BLOCO 9 · SDD

O ciclo SDD



SDD = Spec-Driven Development (Desenvolvimento Orientado a Especificação). A spec é primária; o código é derivado.

► Roteiro de fala

Dois minutos e meio no diagrama central da palestra. O ciclo é: **Intenção** → **Especificação** → **Plano** → **Tarefas** → **Implementação** → **Verificação**. O que muda tudo são os **portões (gates)** entre cada etapa: nada avança sem passar num portão, e em caso de falha, **volta**. Os portões podem ser aprovação humana, teste automatizado, verificação de política ou verificação formal. Agora faça a conexão explícita com o S28: lá vimos que 20 passos não verificados dão 36% de sucesso. **Cada portão aqui corta um elo não verificado dessa cadeia** — é assim, mecanicamente, que se derruba a curva do erro composto. O SDD não é burocracia; é a resposta de engenharia ao erro que se multiplica. (~2,5 min)

Anatomia de uma spec útil

BLOCO 9 · SDD

Anatomia de uma especificação útil

Contexto e escopo · NÃO-objetivos · requisitos em EARS. Dizer o que está fora é tão importante quanto o que está dentro.

Critérios de aceitação verificáveis · contratos · invariantes · referências normativas.

Requisito em EARS (bom):

"Quando uma hemocultura positiva for registrada, o sistema deve avaliar os critérios de ICSAC dentro de 48 h e produzir um parecer com evidência citada."

Requisito ruim (não verificável):

"O sistema deve detectar infecções corretamente."

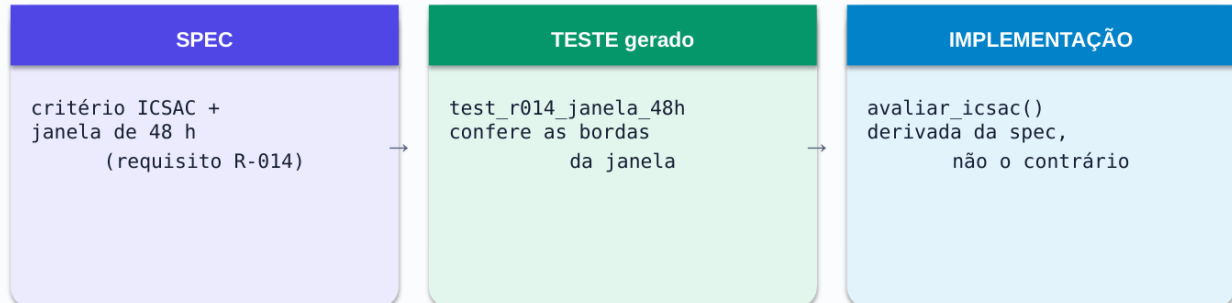
► Roteiro de fala

Dois minutos, e o exemplo faz o serviço. Uma spec útil tem: contexto e escopo, os **não-objetivos** (o que está fora — tão importante quanto o que está dentro), requisitos escritos em **EARS** (uma sintaxe padronizada de requisito), critérios de aceitação **verificáveis**, contratos, invariantes e referências normativas. Mostre o contraste na tela: o requisito bom é *'quando uma hemocultura positiva for registrada, o sistema deve avaliar os critérios de ICSAC dentro de 48 horas e produzir um parecer com evidência citada'* — tem gatilho, prazo e saída verificável. O ruim é *'o sistema deve detectar infecções corretamente'* — impossível de testar. A diferença entre os dois é a diferença entre uma spec que gera teste e uma frase de efeito. (~2 min)

Exemplo ponta a ponta

BLOCO 9 · SDD

Exemplo ponta a ponta: da spec ao código



Rastreabilidade: R-014 → test_r014_janela_48h → avaliar_icsac(). O teste nasceu do CRITÉRIO, não do código — por isso não herda os bugs da implementação.

► Roteiro de fala

Três minutos — é a prova concreta do SDD, então vá devagar. Três painéis. No primeiro, um trecho da **spec**: o critério de ICSAC com a janela de 48 horas, identificado como requisito R-014. No segundo, o **teste gerado a partir do critério** — repare, ele confere exatamente as bordas da janela. No terceiro, a **implementação**, a função `avaliar_icsac()`, que é **derivada** da spec. Agora mostre a linha de **rastreabilidade**: R-014 aponta para o teste, que aponta para a função. E diga a frase que é o coração do método: *'o teste nasceu do critério, não do código; por isso ele não herda os bugs da implementação'*. Um teste escrito a partir do código só confirma o que o código faz — inclusive os erros. Um teste escrito a partir do critério confere o que o sistema **deveria** fazer. (~3 min)

Ferramental

BLOCO 9 · SDD

Ferramental de SDD

GitHub Spec Kit. Comandos `/specify`, `/plan`, `/tasks`, `/implement` — o ciclo do S46 como fluxo de trabalho.

Kiro. Organiza `requirements.md`, `design.md` e `tasks.md` no repositório.

Tessl. Outra aposta no mesmo princípio: a spec como artefato primário.

Feche a promessa do S33: specs/ + AGENTS.md + pacote OKF no MESMO repositório. O que o sistema faz, como se trabalha nele e o domínio que ele modela — tudo markdown versionado.

► Roteiro de fala

Dois minutos, e aqui você fecha um arco que abriu antes do intervalo. O ferramental já existe: o **GitHub Spec Kit**, com comandos que são o próprio ciclo do S46 — `/specify`, `/plan`, `/tasks`, `/implement`; o **Kiro**, que organiza requirements, design e tasks em markdown; e o **Tessl**. Mas o ponto alto é cumprir a promessa do S33: numa árvore de projeto, a pasta **specs/** fica ao lado do **AGENTS.md** e do **pacote OKF** — **o que** o sistema deve fazer, **como** se trabalha nele, e o **domínio** que ele modela, tudo markdown versionado no mesmo repositório. É essa convivência que constrói a rastreabilidade — e a rastreabilidade é o que o próximo slide transforma em argumento comercial. (~2 min)

Onde SDD encontra agentes

BLOCO 9 · SDD

Onde o SDD encontra os agentes



Contexto isolado por etapa, e o GATE em código DETERMINÍSTICO — não em outro modelo. Verificar com um segundo LLM só empilha incerteza; o portão tem que ser código que passa ou não passa.

Feche o S28: o monólito de 20 passos tem 36% de sucesso; a cadeia com gate se comporta de forma inteiramente diferente — o erro não se propaga.

É isto que vou mostrar rodando no Bloco 11: um gerador cujo portão determinístico pega o erro do agente ANTES do deploy.

► Roteiro de fala

Dois minutos e meio — é onde os dois grandes temas da palestra se encontram. Em vez de um agente monolítico fazendo 20 passos, você monta uma **cadeia**: especificador, arquiteto, implementador, verificador — com **contexto isolado** em cada etapa e um **portão entre cada um**. E o detalhe que é a chave de tudo: o portão tem que ser **código determinístico**, não outro modelo de linguagem. Verificar um LLM com outro LLM só empilha incerteza; o portão precisa ser código que passa ou não passa. Faça a matemática do S28 fechar: o monólito de 20 passos tem 36% de sucesso; a cadeia de 4 agentes com portão determinístico se comporta de forma **inteiramente diferente**, porque o erro não se propaga — ele para no portão. E anuncie o Bloco 11: 'vou mostrar isto rodando — um gerador cujo portão determinístico pega o erro do agente antes do deploy'. (~2,5 min)

SDD produz o artefato regulatório de graça

BLOCO 9 · SDD

O SDD entrega o artefato regulatório de graça

Software de saúde exige rastreabilidade requisito → projeto → teste → código. Mais gestão de risco documentada e ciclo de vida controlado: IEC 62304, ISO 14971 e a RDC da ANVISA.

Quem faz SDD JÁ TEM a matriz de rastreabilidade — como subproduto, versionada, com autoria e histórico.

A inversão da objeção: o SDD é o que torna código gerado por IA AUDITÁVEL. O que NÃO é auditável é código escrito à mão sem especificação.

► Roteiro de fala

Três minutos — é o pico comercial da palestra. Depois da última frase, **PAUSE**. Software para saúde exige, por norma, rastreabilidade de requisito até projeto, teste e código, gestão de risco documentada e ciclo de vida controlado — as normas IEC 62304, ISO 14971 e a regulamentação da ANVISA para software como dispositivo médico (confira o número da RDC vigente na semana). Aqui está a virada: **quem faz SDD já tem a matriz de rastreabilidade como subproduto** — versionada, com autoria e histórico, porque ela cai fora do processo naturalmente. Agora inverta a objeção que está na cabeça deles. Eles pensam 'IA generativa em software regulado é risco'. A resposta é: **o SDD é justamente o que torna código gerado por IA auditável**. E o golpe final, diga devagar: *'o que não é auditável é código escrito à mão, sem especificação'*. Complemente com o S30 e o S13: os protocolos não expressam governança e a janela que estoura corta em silêncio — então política de autorização e controle de contexto são responsabilidade da arquitetura de vocês. **Pause depois dessa frase.** (~3 min)

Antipadrões

BLOCO 9 · SDD

Antipadrões: como o SDD morre na prática

Spec inflada. Escreve o que não vai verificar. Antídoto: só entra o que tem critério de aceitação.

Spec envelhecida. Código andou, spec não. Antídoto: CI que falha quando o código diverge da spec.

Gate humano virado carimbo. Aprova sem ler. Antídoto: o revisor assina o diff, não o PDF.

Spec gerada por IA e aprovada sem leitura — a PIOR. Produz aparência de rastreabilidade sem substância. Se ninguém leu, não há SDD, há teatro.

► Roteiro de fala

30 segundos, rápido mas afiado — é onde o sênior vai te testar. Cinco maneiras de o SDD morrer na prática: **spec inflada** (escreve o que não vai verificar; antídoto: só entra o que tem critério de aceitação); **spec envelhecida** (o código andou e a spec não; antídoto: integração contínua que falha quando divergem); **critério não executável**; **gate humano virado carimbo** (aprova sem ler; antídoto: assinar o diff, não o PDF); e a **pior — spec gerada por IA e aprovada sem leitura**, que produz **aparência de rastreabilidade sem substância**. Diga a frase: 'se ninguém leu, não há SDD, há teatro'. (~30 s)

A escada de adaptação

BLOCO 10 · ADAPTAÇÃO

A escada da adaptação

Os degraus, do mais barato ao mais caro: prompt → poucos exemplos (few-shot) → RAG → ajuste fino / LoRA → aprendizado por reforço → pré-treino contínuo.

Só suba um degrau quando o anterior falhar COM EVIDÊNCIA MEDIDA. E a evidência é o conjunto de avaliação do S20 — não o “achismo”.

Retome a seta do S4: é aqui que se faz “o transplante em cima do modelo pronto” que prometi no começo.

► Roteiro de fala

90 segundos para organizar as opções de personalização. Existe uma escada, do mais barato ao mais caro: **prompt**, depois **poucos exemplos** no prompt (few-shot), depois **RAG**, depois **ajuste fino** ou LoRA, depois **aprendizado por reforço**, e no topo **pré-treino contínuo**. A regra de ouro, e martele: **só suba um degrau quando o de baixo falhar com evidência medida** — e a evidência é o conjunto de avaliação do S20. A maioria dos times quer treinar um modelo quando o problema se resolvia com melhor recuperação. Retome a seta do S4: é aqui que se faz o 'transplante em cima do modelo pronto'. (~1,5 min)

SFT e os dados

BLOCO 10 · ADAPTAÇÃO

Ajuste fino (SFT) e os dados

```
{ "messages": [
  { "role": "system",
    "content": "Você classifica critérios de IRAS." },
  { "role": "user",
    "content": "<evolução clínica...>" },
  { "role": "assistant",
    "content": "{ \"critério: ...\", \"atendido: true\" }" }
]}
```

Ajuste fino supervisionado = ensinar por exemplos rotulados. O formato é JSONL: cada linha é uma conversa completa (sistema, usuário, resposta).

500 a 5.000 exemplos BEM curados superam 100 mil sujos. Qualidade vence quantidade — de longe.

O custo real de um dataset clínico é tempo de infectologista, não GPU.

► Roteiro de fala

Um minuto, e a mensagem é contraintuitiva. Ajuste fino supervisionado (SFT) é ensinar o modelo por **exemplos rotulados** — o formato é JSONL, cada linha uma conversa completa. O que a plateia precisa ouvir: **500 a 5.000 exemplos bem curados superam 100 mil exemplos sujos**. Qualidade vence quantidade de longe. E a consequência de gestão: o custo real de um conjunto de dados clínico não é GPU, é **tempo de infectologista** rotulando — planeje por aí. (~1 min)

LoRA e QLoRA

BLOCO 10 · ADAPTAÇÃO

LoRA e QLoRA: ajuste fino que cabe numa GPU

```
LoraConfig(  
    r=16,                # posto: quão "grande" é o remendo  
    lora_alpha=32,  
    lora_dropout=0.05,  
    target_modules=["q_proj", "k_proj",  
                   "v_proj", "o_proj"],  
    task_type="CAUSAL_LM")
```

A ideia: congela o modelo W e treina só um **remendo pequeno** ($\Delta W = B \cdot A$). Treina 0,1–1% dos parâmetros — daí caber numa GPU de consumo.

QLoRA: base em 4 bits + adaptadores em precisão maior. É o que permite ajustar um modelo grande em 24 GB.

Vantagem operacional: adaptadores são arquivos de dezenas de MB — versionáveis e trocáveis em runtime. Um adaptador por especialidade, a MESMA base (pode ser o Qwen3.8-27B do S12).

► Roteiro de fala

Dois minutos e meio — o degrau técnico do bloco. LoRA (adaptação de baixo posto) resolve o problema de custo do ajuste fino: em vez de retrainar o modelo inteiro, você **congela os pesos originais** e treina só um **remendo pequeno** — matematicamente, uma matriz de baixo posto que se soma aos pesos. Você treina de 0,1 a 1% dos parâmetros, e por isso cabe numa GPU de consumo. Explique o trecho: o r é o 'tamanho' do remendo, os *target_modules* são as camadas de atenção que ele modifica. O **QLoRA** vai além: mantém a base em 4 bits e os adaptadores em precisão maior — é o que permite ajustar um modelo grande em 24 GB. E a vantagem operacional que vende a ideia: os adaptadores são **arquivos de dezenas de megabytes**, versionáveis e trocáveis em tempo de execução — um adaptador por especialidade médica, todos sobre a **mesma base**, que pode ser o Qwen3.8-27B do S12. (~2,5 min)

RLHF e sucessores

BLOCO 10 · ADAPTAÇÃO

RLHF e sucessores

RLHF = aprendizado por reforço com feedback humano. Comparações humanas → modelo de recompensa → otimização por reforço (PPO).

Sucessores: DPO (sem modelo de recompensa), GRPO (raciocínio), RLAIF / IA Constitucional.

Por que importa mesmo sem treinar: explica o comportamento do modelo que vocês CONSOMEM. Bajulação, recusa e excesso de ressalvas são artefatos do alinhamento — não do pré-treino.

► Roteiro de fala

Um minuto, e o valor é explicar o modelo que eles já usam. RLHF — aprendizado por reforço com feedback humano — é como se alinha um modelo: humanos comparam respostas, isso treina um **modelo de recompensa**, e o modelo é otimizado para agradá-lo. Os sucessores simplificam: DPO dispensa o modelo de recompensa, GRPO foca raciocínio, o RLAIF usa a própria IA como avaliadora. Mas o motivo de estar na palestra mesmo sem vocês treinarem: **isso explica o comportamento do modelo que vocês consomem** — a bajulação, as recusas exageradas, o excesso de ressalvas são **artefatos do alinhamento**, não do pré-treino. Saber disso ajuda a domar o modelo por prompt. (~1 min)

Quando NÃO treinar, e o modelo local

BLOCO 10 · ADAPTAÇÃO

Quando NÃO treinar, e o modelo local

Não treine com conhecimento volátil (protocolo muda → use RAG), tarefa que muda toda semana, ou sem avaliação. Custos ocultos: esquecimento catastrófico e manutenção perpétua do modelo derivado.

O modelo LOCAL (retomando o S12): quantização (GGUF, AWQ) + runtimes (LM Studio, vLLM, Ollama).

Três argumentos que fecham a palestra técnica: o dado do paciente não sai da rede · custo marginal previsível · independência de fornecedor.

► Roteiro de fala

Um minuto para fechar a adaptação com equilíbrio. Diga quando **não** treinar: se o conhecimento é volátil (um protocolo que muda — use RAG, não ajuste fino), se a tarefa muda toda semana, ou se você não tem avaliação para medir o ganho. E avise dos custos ocultos: o **esquecimento catastrófico** (o modelo perde capacidades ao aprender a nova) e a **manutenção perpétua** do modelo derivado. Feche com o **modelo local**, retomando o S12: quantização e runtimes como LM Studio, vLLM e Ollama permitem rodar em casa. E os três argumentos que importam para saúde: o **dado do paciente não sai da rede**, o custo marginal é previsível, e você não depende de um fornecedor. É a ponte perfeita para o Bloco 11, onde os nossos sistemas rodam exatamente assim — localmente. (~1 min)

Ponte com o domínio deles

BLOCO 11 · SISTEMAS

Ponte: o Co-Scientist e o domínio de vocês

O AI Co-Scientist da Google foi validado em laboratório (Nature, 2026) em três cenários. Repurposing de fármaco para leucemia · alvos para fibrose hepática · e um mecanismo de RESISTÊNCIA ANTIMICROBIANA.

O caso de RAM, nas palavras deles: com pouca informação de fundo, o sistema propôs — sozinho — que os cf-PICs carregam genes de resistência entre espécies, incluindo E. coli e K. pneumoniae.

Isso recapitulou um achado experimental AINDA NÃO PUBLICADO — uma década de bancada. O segundo sistema que vou mostrar foi validado, na origem, exatamente no problema de vocês.

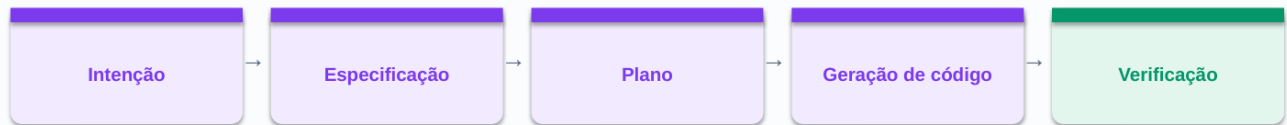
► Roteiro de fala

Um minuto para conectar o segundo sistema ao coração do trabalho deles. O AI Co-Scientist da Google foi validado em laboratório e publicado na Nature em 2026, em três cenários biomédicos: repurposing de fármaco para leucemia, alvos novos para fibrose hepática, e — o que importa aqui — um **mecanismo de resistência antimicrobiana**. E o caso é impressionante para esta plateia: com informação de fundo mínima, o sistema propôs sozinho a hipótese, melhor ranqueada, de que os cf-PICs (ilhas cromossômicas induzíveis por fago) carregam **genes de resistência a antibióticos entre espécies — incluindo E. coli e Klebsiella pneumoniae**, os patógenos da CCIH deles. E isso **recapitulou um achado experimental que ainda não tinha sido publicado**, fruto de uma década de bancada. Diga a frase: *'o segundo sistema que vou mostrar foi validado, na origem, exatamente no problema que vocês enfrentam. Chego nele — antes, o de engenharia.'* (~1 min)

LangNet: arquitetura

BLOCO 11 · SISTEMAS

LangNet: arquitetura



Modelo local no centro: Qwen2.5-Coder-32B via LM Studio, contexto de 64 mil tokens; framework CrewAI. Com avaliação em curso do Qwen3.8-27B — por visão, uso de ferramentas e raciocínio, sem perder geração de código.

Amarre com o S12: o modelo em avaliação roda em 24 GB e é o melhor denso multimodal da faixa. O argumento inteiro do LangNet só existe porque essa faixa chegou a esse patamar em 2026.

► Roteiro de fala

Três minutos apresentando o primeiro sistema. O LangNet implementa o ciclo do S46: da intenção à especificação, ao plano, à **geração de código** e à **verificação** — com um **modelo local no centro**. A pilha real, e diga que confere com o repositório: **Qwen2.5-Coder-32B rodando no LM Studio, com 64 mil tokens de contexto**, framework CrewAI, e uma **avaliação em curso do Qwen3.8-27B** como substituto — motivada por visão, uso de ferramentas e raciocínio num modelo menor, sob o critério de não perder capacidade de geração de código. Amarre com o S12: o modelo que estamos avaliando saiu há semanas e é o melhor denso multimodal que roda em 24 GB — **o argumento inteiro do LangNet só existe porque essa faixa de hardware chegou a esse patamar em 2026**. Isso é evidência de método, não de sorte. (~3 min)

LangNet: evidência medida

BLOCO 11 · SISTEMAS

LangNet: evidência medida

Cobertura de rastreabilidade: 100%. Todos os requisitos funcionais atravessam Spec → Modelo de Dados → Implementação, medido por PORTÃO DETERMINÍSTICO — não por confiança no modelo.

Suíte de tarefas geradas: 100% executando ponta a ponta. Contra banco real — núcleo determinístico + tarefas com agente (laudo, extração de documento).

~10 defeitos do gerador capturados pelo portão ANTES do deploy. Variável indefinida, coluna inexistente, junção espacial faltante — cada um com o salto exato onde quebrava.

O número não é a taxa de acerto do modelo — é a taxa em que o portão **pega o erro antes de virar produção**. É o S50 acontecendo. (Remeça no exemplo clínico do vídeo.)

► Roteiro de fala

90 segundos — e é o número que separa a apresentação de um pitch, então diga com firmeza. São medições reais da validação do LangNet, não estimativa. **Cobertura de rastreabilidade: 100%** — todos os requisitos funcionais atravessam da especificação ao modelo de dados e à implementação, e isso é medido por um **portão determinístico**, não por confiança no modelo. **Suíte de tarefas geradas: 100% rodando ponta a ponta** contra um banco real, incluindo tarefas determinísticas e tarefas com agente. E o mais importante: **cerca de dez defeitos do gerador foram capturados pelo portão antes do deploy** — variável indefinida, coluna que não existe, junção espacial faltando — cada um com o ponto exato onde quebraria. Diga a frase que reposiciona o número: *'isto não é a taxa de acerto do modelo; é a taxa em que o portão pega o erro do modelo antes de virar produção'* — é o S50 acontecendo de verdade. Nota: remeça esse número no exemplo clínico que for ao vídeo, para casar com o domínio deles. (~1,5 min)

DEMO gravada: da especificação à aplicação

BLOCO 11 · SISTEMAS

DEMONSTRAÇÃO GRAVADA: da spec à aplicação

📺 VÍDEO — até 3 min, sem áudio, legendas grandes



(inserir a gravação da tela do LangNet)

- 1. você escreve uma spec curta (15–20 linhas) — deixe na tela para ler
- 2. geração do plano e das tarefas
- 3. app completo: modelo de dados, lógica, API e **interface**
- 4. execução da suíte derivada dos critérios
- 5. uma falha de teste e a correção — **não corte**
- 6. a aplicação rodando com a interface
- 7. volta à spec, com o **portão de rastreabilidade VERDE**

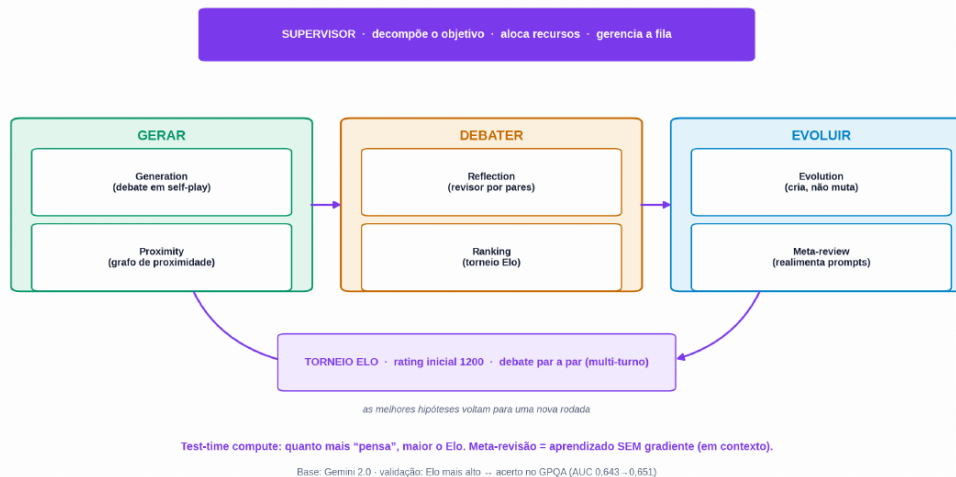
► Roteiro de fala

Três minutos — é o ponto mais alto de prova da palestra. Deixe o vídeo falar. Toque a gravação (até 3 minutos, sem áudio, legendas grandes; acelere as esperas de 2 a 4 vezes com o tempo real na legenda). O roteiro do vídeo, que mapeia um-para-um o que o LangNet faz: (1) você escreve uma spec curta, de 15 a 20 linhas — **deixe na tela tempo de ser lida**; (2) ele gera o plano e as tarefas; (3) gera o **aplicativo completo** — modelo de dados, lógica, API e **interface**; (4) roda a suíte de testes derivada dos critérios; (5) **uma falha de teste e a correção — não corte essa parte**, é o que prova que é real; (6) a aplicação rodando com a interface; (7) volta à spec, com o **portão de rastreabilidade verde na tela** — fechando o arco do S48 e do S51. Ao fim, 30 segundos seus: diga o tempo real, os tokens gastos e o que você corrigiu à mão. (~3 min)

Como funciona o AI Co-Scientist

BLOCO 11 · SISTEMAS

Como funciona o AI Co-Scientist (Nature, 2026)



► Roteiro de fala

Dois minutos e meio no sistema multiagente mais elegante que existe hoje. É da Google DeepMind, sobre o Gemini 2.0, publicado na Nature em 2026. O objetivo não é 'pesquisa profunda' — é **gerar hipóteses novas e testáveis** a partir de um objetivo em linguagem natural. Percorra o diagrama: um **Supervisor** decompõe o objetivo e gerencia a fila; e seis agentes trabalham em três fases. **Gerar**: o Generation propõe hipóteses via debate consigo mesmo, o Proximity as agrupa num grafo. **Debater**: o Reflection critica como revisor por pares, e o Ranking roda um **torneio Elo** — cada hipótese entra com nota 1200 e disputa par a par em debate. **Evoluir**: o Evolution refina as melhores (e cria novas, em vez de mutar), e o Meta-review **realimenta os prompts** das rodadas seguintes. Agora os quatro insights que valem mais que a lista: (1) o Elo é a **função de aptidão** do sistema; (2) o grafo decide quais duelos valem a pena; (3) a evolução **cria em vez de mutar**, para não destruir o que já passou no torneio; (4) a meta-revisão é **aprendizado sem gradiente** — o sistema melhora sem ajustar um único peso, só realimentando o contexto. Amarre com o S8 (é processamento na resposta), o S25 (reconheçam os padrões: avaliador-otimizador, votação, orquestrador) e o S39 (é o AutoGen com uma função de aptidão explícita — a diferença é o Elo). (~2,5 min)

AI Co-Scientist: a sua replicação

BLOCO 11 · SISTEMAS

A nossa replicação: QuanticaResearch

O que MANTIVEMOS do paper (no código). Os 6 agentes + Supervisor em LangGraph; torneio Elo (nota inicial 1200, K=32), debate multi-turno guiado pelo grafo de proximidade; meta-revisão realimentando os prompts.

O que MUDAMOS, e por quê (aqui está a engenharia). LLM agnóstico de fornecedor — inclusive LOCAL (Ollama, LM Studio) — no lugar do “só Gemini”; fila durável própria (FastAPI + MariaDB) no lugar do agendador do Google; grafo de proximidade com NetworkX + Louvain.

Resultado exercitado (run UC-10). Rodou ponta a ponta: revisão de literatura + 4 hipóteses + 1º torneio + 1ª meta-revisão, em ~2,8 h com DeepSeek.

Honestidade: réplica funcional PRONTA PARA PILOTO, não produção. A sincronização com o banco ainda tem um bug conhecido (alfa).

► Roteiro de fala

Dois minutos mostrando que isto não é slide de PowerPoint — é sistema que roda. O QuanticaResearch é uma plataforma full-stack (React, FastAPI, MariaDB) sobre um motor multiagente em LangGraph. Estructure a fala em três partes — o que você **manteve**, o que **mudou**, e o **porquê** — porque o porquê é o que demonstra engenharia, não reprodução. **Mantivemos** os seis agentes e o supervisor, o torneio Elo com nota inicial 1200, o debate par a par multi-turno guiado pelo grafo de proximidade, e a meta-revisão realimentando os prompts. **Mudamos**, e é aqui que está o trabalho: o paper é **exclusivamente Gemini** e roda no agendador interno do Google; a nossa camada de modelo é **agnóstica de fornecedor e roda inclusive local** (Ollama, LM Studio), a fila é durável e própria (FastAPI mais MariaDB), e a proximidade usa NetworkX com detecção de comunidades. E há resultado: o experimento UC-10 rodou **ponta a ponta** — literatura, quatro hipóteses, torneio e meta-revisão — em cerca de 2 horas e 48 minutos com o DeepSeek. Seja honesto sobre a maturidade: o motor roda fim a fim, mas a sincronização com o banco ainda tem um bug conhecido; apresente como **réplica pronta para piloto, não produção**. Isso é credibilidade, não fraqueza. (~2 min)

Validação por Redes de Petri

BLOCO 11 · SISTEMAS

A camada que amarra tudo: Redes de Petri

VisualTasksExec: sincroniza agentes com propriedades VERIFICÁVEIS. Ausência de deadlock (travamento), alcançabilidade, invariantes de estado — provados na estrutura, não testados por amostragem.

Feche o arco inteiro da palestra: o S28 mostrou que agentes degradam de forma previsível; o S50, que gates cortam a cadeia; o S30, que nenhum protocolo expressa governança; o S62, um sistema que se autoavalia por Elo.

Mas o Elo mede qualidade de HIPÓTESE, não corretude de EXECUÇÃO. As Redes de Petri mostram que a estrutura da orquestração pode ser VERIFICADA FORMALMENTE — não apenas testada ou pontuada.

► Roteiro de fala

Dois minutos e meio para fechar o arco técnico da palestra inteira — deixe o argumento falar, não vire pitch. As Redes de Petri são um formalismo matemático para modelar processos concorrentes. O VisualTasksExec usa isso para sincronizar agentes com **propriedades verificáveis**: ausência de travamento (deadlock), alcançabilidade de estados, invariantes — coisas que você **prova na estrutura**, não descobre testando. Agora costure o arco inteiro: o S28 mostrou que agentes degradam de forma previsível; o S50, que os portões cortam a cadeia; o S30, que nenhum protocolo do mercado expressa governança; o S62, um sistema que se autoavalia por Elo. Mas — e aqui está o remate — **o Elo mede qualidade de hipótese, não corretude de execução**. Este slide mostra que a estrutura da orquestração pode ser **verificada formalmente**, não apenas testada ou pontuada. É o nível mais alto de garantia, e é onde a engenharia de vocês pode ir além do estado da arte. (~2,5 min)

As três conclusões

BLOCO 12 · FECHAMENTO

As três conclusões

① **A capacidade virou mercadoria comum; o método, não.** O que separa os times é especificar e verificar — não o acesso ao modelo.

② **Agente sem portão degrada de forma previsível.** Confiabilidade vem de reduzir passos não verificados (S28, S50).

③ **A especificação executável é o que torna código de IA auditável.** E, em software de saúde, entrega o artefato regulatório de graça (S51).

Recomendação acionável, uma só: construam PRIMEIRO o conjunto de avaliação (S20). Antes de framework, antes de agente, antes de trocar de modelo.

► Roteiro de fala

90 segundos de fechamento — retome a tese do S3 e amarre tudo. Três conclusões: (1) a capacidade de gerar código virou **mercadoria comum**; o que diferencia um time é o **método**. (2) Um agente sem portão de verificação **degrada de forma previsível** — e confiabilidade vem de reduzir os passos não verificados. (3) A **especificação executável** é o que torna código de IA auditável, e em software de saúde ela entrega o artefato regulatório de graça. E então dê a eles **uma única recomendação acionável**, para saírem da sala sabendo o que fazer na segunda-feira: **construam primeiro o conjunto de avaliação do S20** — antes de escolher framework, antes de montar agente, antes de trocar de modelo. Sem esse conjunto, todo o resto é chute. (~1,5 min)

Referências e contato

BLOCO 12 · FECHAMENTO

Referências e contato

Papers e specs. Co-Scientist: Gottweis et al., Nature 2026 (s41586-026-10644-y) / arXiv 2502.18864. Specs: MCP, A2A, OKF, GitHub Spec Kit.

Modelos. A tabela do S12, com os links — reconferida na semana da palestra.

Repositórios e contato. QR do repositório do LangNet e do QuanticaResearch; seu e-mail e perfil.

► Roteiro de fala

30 segundos. Deixe o slide de referências no ar enquanto respira: um QR para o repositório, os papers (o Co-Scientist na Nature), as especificações (MCP, A2A, OKF, Spec Kit), a tabela de modelos do S12 com os links, e o seu contato. Avise que os slides e o roteiro ficam disponíveis. (~30 s)

Perguntas

Perguntas

“O gargalo deixou de ser escrever código. Passou a ser especificar e verificar.”

Obrigado.

► Roteiro de fala

Três minutos e meio de perguntas — e você tem o Anexo A do planejamento com as respostas curtas prováveis. Deixe a tese no rodapé do slide como âncora. Se a plateia travar, provoque com uma das perguntas frequentes (qual modelo aberto usar, CrewAI ou LangGraph, contexto de 1 milhão resolve o prontuário). Feche agradecendo e lembrando que os slides e o roteiro ficam disponíveis. (~3,5 min)